



HACKTHEBOX



Ancient Interface

15th April 2023 / Document No. D23.102.11

Prepared By: w3th4nds

Challenge Author(s): ryagaard

Difficulty: **Medium**

Classification: Official

Synopsis

Ancient interface is a medium difficulty challenge that features `SIGALRM` interrupting `read` `syscall` causing it to return a negative amount resulting in a buffer underflow.

Skills Required

- Linux syscalls
- Linux Signal

Skills Learned

- SIGALARM, read syscall, buffer underflow.

Enumeration

The one command that stands out from the rest is `alarm`, as it is not an actual command in `bash/sh`. We can also see in `cmd_read`, on line 253, that the return value of the `read` syscall is unchecked, which means, that if it returns a negative amount `read_bytes` will decrement, and on the following loop iteration, the program will go backwards the buffer argument for `read` call.

```
read_bytes = 0;
do
{
    read_bytes += read(0, buf + read_bytes, amnt - read_bytes);
} while (read_bytes != amnt);
```

To make `read` return a negative value we can use the `alarm` command, as it says on the [man page](#).

If a `read()` is interrupted by a signal before it reads any data, it shall return -1 with `errno` set to `[EINTR]`.

Achieving a buffer underflow scenario is not hard from here. We can send alarm 5 to the program, which should send a `SIGALRM` after 5 seconds, and before those 5 seconds have elapsed, we send read 8 whichever commands and wait for the alarm signal.

While waiting for the alarm `read_bytes` value to become 0, and after the program receives the signal, `read` returns -1, where the program does `read_bytes += -1`, effectively making `read_bytes` value be -1. And as `read_bytes` is not equal to the amount, it goes into the loop again, except this time it will call `read(0, buf + -1, amnt - -1)`, which means it will read into `buf -1` causing a buffer underflow.

In order to reach the next return address below the buffer, we would need to go `0x28` bytes below it.

where ``read`` will start reading bytes

```

      |
      v
0x7fff68f0ae28: 0x000000000000401a1a      0x0000000000000000
0x7fff68f0ae38: 0x000007fff68f0be95      0x00000100ffffffd8
0x7fff68f0ae48: 0x000007fff68f0be98      0x0000000000000000      <--- char
buf[4096]
0x7fff68f0ae58: 0x0000000000000000      0x0000000000000000
0x7fff68f0ae68: 0x0000000000000000      0x0000000000000000
0x7fff68f0ae78: 0x0000000000000000      0x0000000000000000
0x7fff68f0ae88: 0x0000000000000000      0x0000000000000000
0x7fff68f0ae98: 0x0000000000000000      0x0000000000000000
```

`0x7fff68f0ae28` is the address where the program will read input from the user after we call `0x28` alarms to go `0x28` bytes below it. `0x000000000000401a1a` is the return address for the `read` call which, we will overwrite and start our ROP chain. As the program doesn't have PIE, we don't need any leaks to proceed with the ROP chain.

In the exploit we call `write(1, &strchr@got, 0x100)` to leak the address of `strchr` to leak `libc` address, and then within the same ROP chain right after the write call, we call `read(0, &strchr@got, 0x100)`, so when we get a `libc` leak, we can overwrite `strchr@got` with the address of `system` from `libc`. After all of that, we make the program jump to `0x000000000000401bbb`, which is the address where `main` calls `strchr`.